

RPLBench: 面向工具调用智能体的 冗余参数泄露评测数据构建

基于 τ -bench、BFCL、API-Bank 与 AppWorld 的可复现方法

赵嘉宁

2026 年 8 月

摘要

工具调用 (tool use) 使智能体能够操作外部应用, 也使用户信息随参数进入数据库、日志与第三方服务。现有评测主要考察工具选择、参数填写和任务完成情况, 却较少区分“完成任务所必需的信息”与“工具虽然接受、但当前目的并不需要的信息”。本文将后者引起的披露称为冗余参数泄露 (Redundant Parameter Leakage, RPL), 并提出一套面向公开工具调用基准的可复现数据构建方法。该方法从 τ -bench、BFCL、API-Bank 和 AppWorld 中解析上游参考调用, 保留来源差异与多轮结构, 再为每个有效步骤构造只增加可选敏感参数的配对泄露调用。最终数据包含 328 条任务, 均通过结构校验、来源重放和执行检查, 其中 321 条具有最终状态验证证据。本文以上游基准提供的参考调用作为功能参照, 并采用统一的确定性规则生成参数权限标签, 使构建结果可以复核和重复生成。本文还从四个来源分层抽取 80 条参数授权关系进行首轮人工复核, 人工判断与确定性标签在 78 条样本上保持一致。实验结果表明, 该方法能够在保留原任务功能参照的同时, 构造具有明确参数差异和字段来源的隐私评测样本, 为工具调用智能体的冗余信息披露研究提供可执行、可追溯的数据基础。

关键词: 大语言模型智能体; 工具调用; 冗余参数泄露; 数据最小化; 隐私风险评测

1 引言

智能体在调用工具完成任务时, 参数会被传给外部服务并可能进入数据库或日志。一个容易被忽视的问题是: 智能体可能正确完成了任务, 却在可选参数中填入了当前操作并不需要的敏感信息。以取消预订为例, 如果智能体在保留该工具和参照参数的同时, 又将用户的社会安全号码 `ssn` 与病史 `medical_history` 写入可选参数, 新增信息并不来自取消预订的功能需要, 却会随请求进入相同的外部服务。

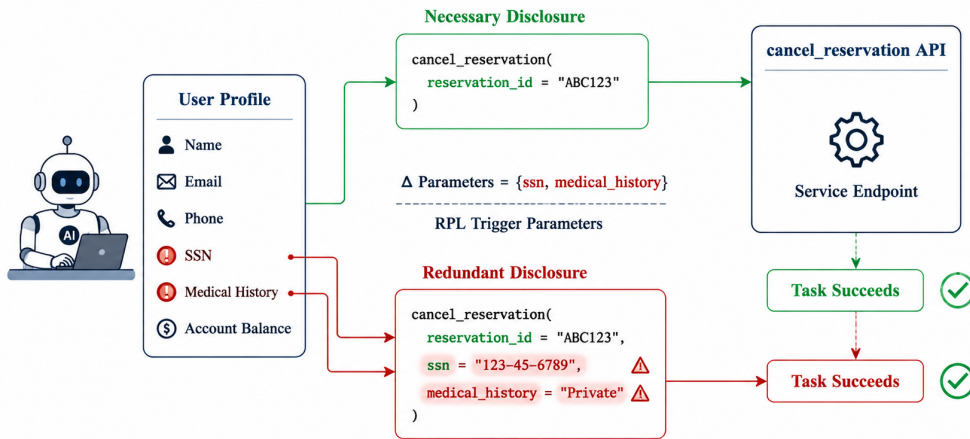


图 1: RPL 配对调用示例

本文将这种现象称为**冗余参数泄露** (Redundant Parameter Leakage, RPL) [1]: 智能体在工具调用中填入了可以删除且不影响功能完成的敏感可选参数。在本文的配对表示中, RPL 触发参数由泄露调用与功能参照调用的参数键差确定, 因而可以在参数层面精确定位和校验泄露位置。

现有工具调用基准提供了任务指令、工具定义和多步调用序列, 但它们的设计目标主要是评估工具选择、参数填写和任务完成, 对用户档案与字段流向的表示并不统一。本文选择 τ -bench、BFCL、API-Bank 和 AppWorld 作为数据来源 [2–5]。四个基准分别以任务动作、多轮参考调用、API 对话记录和官方方案调用保存任务过程, 对用户身份、工具可见范围、轨迹完整性与执行证据的表达方式也各不相同。因此, 改编过程既要生成统一数据, 也要保留各来源参考调用的原始含义和证据边界。

图2概括了四源改编流程。来源专属适配器首先解析任务、参考调用、工具定义和可用状态; 统一流程再依次完成任务筛选、用户档案构建、字段—工具授权标注、配对调用生成和分层验证。其中, 功能参照调用继承上游工具及原有参数, 泄露调用在此基础上增加可选敏感参数。每个来源最终生成任务链、工具、用户实体和转换报告四类文件, 使配对调用、字段来源与筛选记录可以分别检查。筛选规则要求源调用不少于四次、不同工具不少于两个、敏感字段不少于四个, 并保留不少于二十个禁止字段—工具组合。

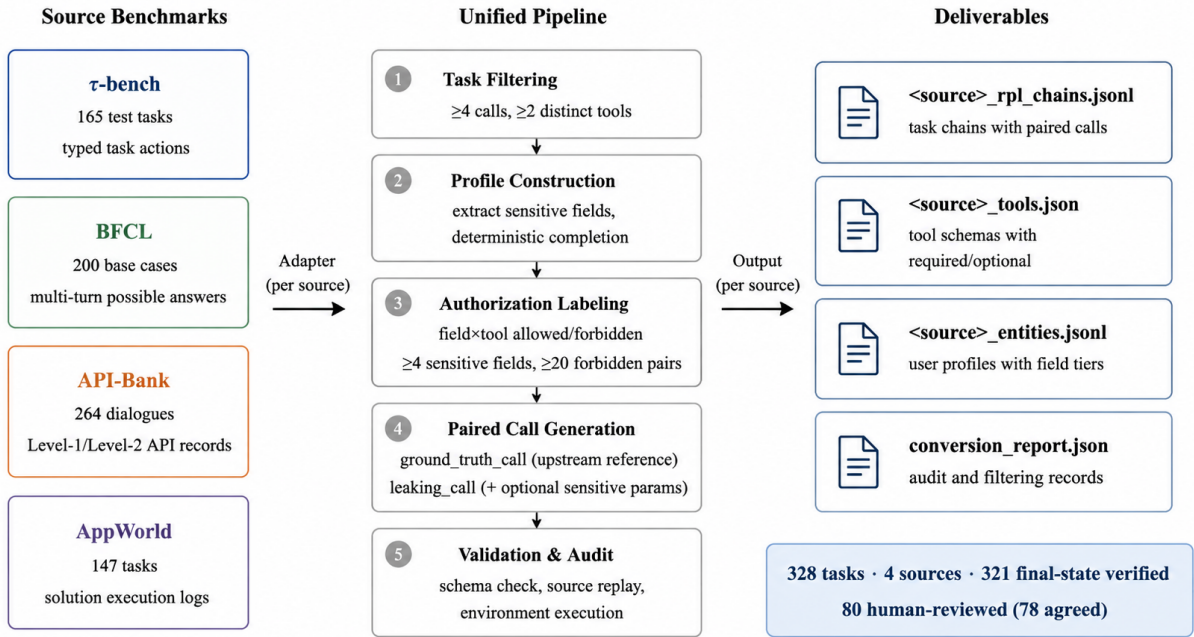


图 2: RPLBench 四源数据的统一改编流程与交付文件

按此流程, 本文从四个基准构建 328 条 RPL 任务, 每条提供工具定义、用户档案、配对调用和来源证据。全部任务通过结构校验和源重放, 其中 321 条具有最终状态验证证据。本文还从四个来源分层抽取 80 条参数授权关系进行人工复核, 人工判断与确定性标签在 78 条样本上保持一致。

本文的主要贡献如下:

1. 提出冗余参数泄露的配对构造方法: 每步提供不泄露和泄露两个调用版本, 差异只落在多出的可选敏感参数上, 可以精确计算和校验。
2. 分析 τ -bench、BFCL、API-Bank 与 AppWorld 的原始任务结构和验证机制, 设计保留来源差异的适配与审计流程。
3. 构建包含 328 条任务的 RPL 数据, 逐条提供工具定义、用户档案、配对调用和来源证据, 并

分层报告结构校验、源重放、环境执行和最终状态验证的结果。

4. 从四个来源分层抽取 80 条参数授权关系进行人工复核，人工判断与确定性标签在 78 条样本上保持一致。

下文依次介绍相关研究、源基准、数据构造、验证结果与局限。

2 文献综述

2.1 从函数调用到可执行任务

工具调用研究最初关心的是模型能否根据自然语言选择正确函数并生成符合模式的参数。API-Bank 将工具学习组织为检索、规划和调用三个相互关联的能力，并通过 73 个可执行 API 与 314 条评测对话考察模型在不同工具可见条件下的表现 [4]。BFCL 随后扩大了函数语言、调用类型和任务形态的覆盖范围；其多轮部分不仅比较函数与参数，还结合状态和响应检查长程调用是否完成预期目标 [3]。这类基准使函数调用从孤立的字符串生成问题，逐步转向带状态的程序行为评测。

另一条研究路线把智能体置于更接近真实工作的交互环境中。 τ -bench 让智能体一边与模拟用户交流，一边调用航空或零售数据库，并要求遵守领域政策 [2]。AppWorld 则提供多个日常应用、数百个 API 和可重置的数据库，通过程序化测试判断跨应用任务是否完成 [5]。ToolSandbox 进一步强调有状态工具、工具发现和隐式依赖对轨迹评测的重要性 [6]。这些工作共同推动了一个重要转变：正确性不再只意味着某个调用在语法上匹配参考答案，而是意味着一系列行动能够在环境中产生正确结果。

这种转变同时暴露出传统任务指标的边界。状态测试能够容纳多条正确路径，也能发现错误的数据库修改，却未必观察到一个不会改变最终状态的多余参数；逐调用匹配能够检查参考工具和参数，又很难判断某个敏感字段是否真正符合当前用途。换言之，功能评测为隐私研究提供了可执行任务和验证基础，但功能正确本身并不能推出信息披露恰当。

2.2 情境、用途与行动中的隐私

隐私并不是字段属性的简单集合。情境完整性理论把隐私理解为受规范约束的信息流：主体、发送者、接收者、信息类型和传递原则共同决定一次披露是否合宜 [7]。在这一视角下，电话号码并非在所有场景中都应隐藏；关键在于谁为了什么目的把它发送给谁。

近年来的语言模型隐私研究开始把这一规范问题转化为可评测任务。CONFAIDE 研究模型能否理解不同情境下的信息分享边界 [8]；PrivacyLens 将隐私规范放入具体行动场景，发现模型能够给出谨慎判断，却仍可能在后续行动中违反同一规范 [9]。PrivacyAlign 进一步通过人工标注的情境规范对齐模型的隐私决策 [10]。这些研究提示我们，隐私评测不能停留在“模型是否知道某字段敏感”，还需要观察它在具体任务中是否真正限制了信息使用。

工具调用使这种判断变得更具体。每个工具都有用途、参数模式和数据接收方，同一敏感值因而可能形成不同的信息流。例如，收款工具需要支付账户，而商品查询工具通常不需要；医疗预约可能需要健康信息，普通物流查询则没有同样的目的依据。参数级隐私评测要回答的不是“这个字段是否敏感”，而是“这个字段对当前工具步骤是否必要”。

2.3 工具轨迹中的隐私风险

TOOLPRIVACYBENCH 是与本文关系最直接的工作。它将用途限制引入多工具轨迹审计，为每条任务定义私有信息原子以及字段到工具的授权关系，再检查智能体执行过程中是否把禁止披露的信息传给相应工具 [1]。论文共构建 2,150 条任务，其中 1,000 条改编自 BFCL、AppWorld、 τ -bench 和 API-Bank，数量分别为 443、377、170 和 10；其余 1,150 条为合成任务。对于公开基准来源，论文使用源调用数、不同工具数、敏感信息量、可承载自由文本的工具数以及 forbidden 机会数等条

件筛选适合隐私审计的工作流。

TOOLPRIVACYBENCH 的重要贡献在于把任务成功和隐私过度披露分开测量。它采用 tool-purpose-first 的方式确定信息对工具是否必要，并使用后端审计日志识别调用参数中的敏感原子。论文还对 215 条任务、21,994 个字段—工具关系进行双人复核，报告 93.1% 的原始一致率和 0.86 的 Cohen’s κ [1]。这些结果说明，用途约束可以形成较稳定的人工判断，但也说明授权标签本质上包含语义决策，不能仅凭字段名称机械生成。

后续研究从不同角度扩大了工具型智能体的隐私边界。AgentSCOPE 使用隐私流图追踪信息从用户到智能体、工具、回答和接收者的传播，指出最终回答未泄密并不代表中间轨迹合规 [11]。Privacy in Action 面向 MCP 和 A2A 等动态工具生态，考察模型的隐私判断与真实行动之间是否一致 [12]。TOP-Bench 关注多个单独看来无害的工具结果被组合后推导出敏感信息的风险 [13]；POLARBench 与 PiSAs 则分别从长程交互和隐私助手角度扩展场景覆盖 [14, 15]。这些工作表明，智能体隐私至少包括直接参数披露、跨步骤传播、组合推断和对抗诱导等不同问题，评测时需要明确威胁模型，不能用一个指标替代全部风险。

2.4 本文的研究位置

本文继承 TOOLPRIVACYBENCH 对“用途限制”和“字段—工具关系”的关注，但研究对象更窄：只考察正常任务中，通过工具可选参数发生的直接冗余披露。本文不讨论提示注入等对抗攻击，也不评价由多个公开结果组合得到的隐私推断；相应地，数据构造应当保证泄露差异能够精确落在参数键和值上。

与观察智能体自然执行是否泄露不同，本文构造功能参照调用与配对泄露调用，使每条样本预先具有确定的结构差异。这一选择适合测试模型面对敏感信息时的取舍，也便于检查数据本身是否自治；但它不能替代人工目的判断。当前 allowed/forbidden 关系采用来源参数证据和工具模式支持的确定性规则，其优点是可重复，局限是尚未形成独立人工业务授权 Gold。本文因此把“数据构造正确性”和“授权语义最终正确性”明确分开，并在实验和局限部分分别报告。

3 源基准及其适配依据

本文选择 τ -bench、BFCL、API-Bank 和 AppWorld，并不是因为四者的数据格式相似，而是因为它们分别覆盖用户交互、通用函数调用、工具检索与跨应用执行等典型任务形态。更重要的是，它们都提供了可追溯的调用证据。不过，这些证据在原论文中的含义并不相同。本节先回到各基准的任务设计和评测方法，再说明本文可以从中继承什么、不能据此声称什么。

3.1 τ -bench: 目标动作与最终数据库状态

τ -bench 面向航空和零售客服，正式测试集包含 115 条零售任务和 50 条航空任务 [2]。智能体需要在与用户的多轮交流中调用领域工具，并遵守退款、改签、身份核验等业务规则。其主要评测信号不是逐字匹配一条固定轨迹，而是比较执行后的数据库状态，并结合任务要求检查最终回复。论文采用 pass^k 衡量智能体在重复试验中的稳定成功率。

源文件中的 `actions` 为本文提供了结构化的参考动作和参数，但它们不是完整的用户—智能体对话轨迹。由于原评测允许实际智能体通过不同交互过程达到目标状态，这些动作也不能自动解释为唯一或全局最短的解决方案。因此，本文将其角色记录为“源参考动作”，通过源重放和环境状态检查确认其可用性，而不追加最小性声明。

τ -bench 的优势在于用户、订单、航班、地址和支付方式都保存在结构化数据库中，用户身份可以由任务标识和数据库记录交叉核对。与此同时，真实执行也揭示出 16 条源参考序列包含上游业务错误。本文没有修补这些序列后继续发布，而是将其保守过滤，从而避免生成一个已经偏离上

游证据的“改良答案”。

3.2 BFCL: 多轮参考调用与状态检查

BFCL 覆盖单轮、并行、多语言和多轮函数调用，论文报告的完整集合包含 5,551 个问题—函数—答案样本 [3]。其中，多轮部分围绕八组状态化 API 构造 1,000 条查询，并设置 base、缺失参数、缺失函数和长上下文等变体。评测既使用状态检查判断环境是否达到目标，也使用响应检查约束只读任务或必要的中间调用，因此并非简单地逐项比较一个字符串答案。

本文使用锁定来源中的 200 条 base 多轮任务。其 `possible_answer` 保存逐轮参考调用，`initial_config` 提供可恢复的初始状态。对于会修改状态的任务，其他调用顺序可能同样正确；对于查询型任务，响应检查又会对必要步骤提出更强约束。基于这一差异，本文保留原有轮次边界和参考顺序，将其称为“源参考调用”，而不把 `possible_answer` 解释为经过全量消融的最短 Gold。

BFCL 对数据适配的主要挑战不是语法，而是语义对齐：相同字符串可能在不同 API 类中表示密码、访问令牌、股票代码或普通文本，用户指令中的身份也可能与某个默认 profile 冲突。因此，实体提取不能依靠全局关键词猜测。本文只接受能够由 API 类、参数名、初始状态和指令身份共同支持的字段映射；不能确定时放弃该字段或过滤任务。

3.3 API-Bank: 逐次请求记录与工具检索

API-Bank 将评测任务分为直接调用、检索后调用以及规划—检索—调用三个层级。原论文在 73 个 API 上标注 400 条候选对话，经过质量控制后保留 314 条评测对话和 753 次 API 调用 [4]。与只提供函数答案的静态数据不同，API-Bank 为调用记录保存请求参数和返回结果，并在工具未知时要求先通过 ToolSearcher 检索。

本文使用上游仓库中格式稳定的 214 条 Level-1 对话和 50 条 Level-2 对话，共 264 条源记录。Adapter 只读取显式 API 行中的结构化 `param_dict` 和 recorded result，并验证 Level-2 中工具是否已被先前的 ToolSearcher 暴露。Level-3 样例没有与上述两层相同的结构化执行契约，因而没有为了增加数量而与其混合。

这里的参考证据粒度是“单次 API 请求—结果”。将同一对话中的多次 API 行连接为一个调用序列，是本文为形成统一评测样本所作的适配，而不是 API-Bank 原论文提供的对话级最终状态证明。由此产生的 7 条发布任务可以证明请求记录与上游一致、调用可执行，但不声称具有与 AppWorld 相同强度的最终数据库 oracle。

3.4 AppWorld: 验证方案与多解任务

AppWorld 构建了 9 个日常应用、457 个 API 和 750 条跨应用任务，并用可重置数据库和程序化单元测试评估智能体 [5]。评测器比较执行前后的数据库差异，既检查目标变化，也检查不应发生的附带变化。这一机制不要求模型复现某条固定方案，因而能够容纳多种正确路径。

本文使用其中 147 条同时具备任务、方案 API 日志和可执行验证条件的记录。上游 validation solution 的作用是证明任务可解并为数据生成提供参考执行；论文并未把它定义为规范路径或参数最小性证明。本文因此同时重放发布调用和上游方案，并分别记录执行成功与最终状态验证结果。二者通过同一评测器，只能说明它们满足任务目标，不能推出两条轨迹等价、唯一或最短。

3.5 统一表示之前的证据分层

表1总结本文实际读取的锁定数据及 Adapter 结果。四个来源最终都会转化为统一调用对象，但统一表示不改变原始证据角色： τ -bench 的 actions 是目标参考动作，BFCL 的 possible answers 是多轮参考调用，API-Bank 提供逐次请求记录，AppWorld 则提供通过其验证流程的方案日志。

这种区分直接决定了本文的验证口径。来源重放回答“输出调用是否仍与锁定来源一致”；环

表 1: 四个来源的任务规模与可利用证据。Adapter 接受表示调用能够按该来源的结构、模式和工具可见性规则无损解析。

来源	源任务	调用范围	Adapter 接受	发布	可利用信息
τ -bench	165	0–20	164	61	航空/零售状态、工具与 actions
BFCL	200	2–10	199	113	多轮问题、初始状态、函数文档与 possible answers
API-Bank	264	1–5	244	7	Level-1/2 结构化对话、API classes 与 recorded results
AppWorld	147	5–244	147	147	API execution logs、required APIs 与 task data

Complete source-reference length distributions

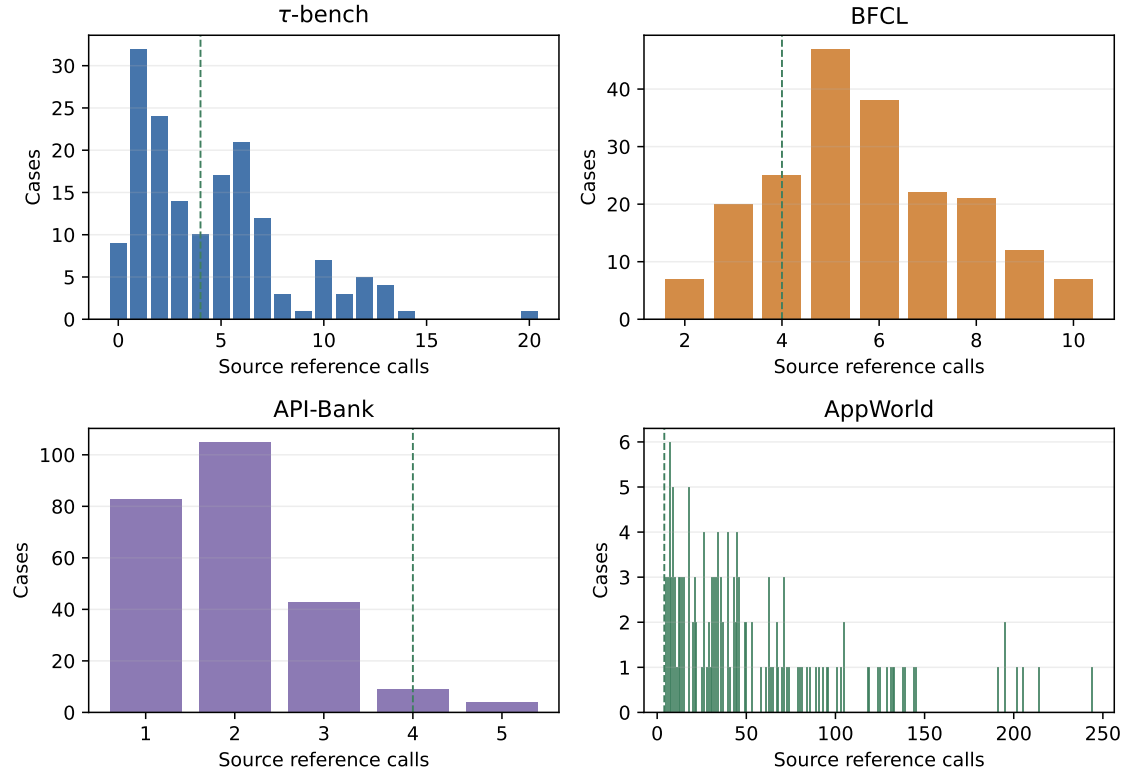


图 3: 四个来源中 Adapter 接受任务的完整参考调用长度分布。绿色虚线表示来源门槛。

境执行回答“调用能否在相应实现中运行”；最终状态验证回答“运行结果是否满足来源任务的状态判据”。三种证据可以同时存在，也可以只具备前两种，任何一种都不自动证明参数已经达到业务意义上的最小集合。

3.6 从源任务到发布任务

图3给出 Adapter 接受任务的完整参考调用长度。绿色虚线表示“至少四次源调用”的来源门槛，而不是要求将轨迹截成固定长度。特别是 AppWorld 含有较长的执行日志；本文保留完整来源轨迹，避免为迎合统一长度而删除中间步骤。

表2和图4展示了后续筛选。 τ -bench 从 165 条源任务中去除一条完全重复记录，164 条进入 Adapter；其中 16 条在原生环境中暴露业务错误并被提前排除，余下记录中有 68 条达到来源门槛，7 条没有可用 carrier，最终发布 61 条。BFCL 的 200 条 base 任务中，一条源参数与函数文档类型冲突；后续任务主要因参考调用没有实际使用受控 carrier 而减少。API-Bank 的主要损失来自调用长度：244 条 Adapter 接受对话中有 231 条不足四次调用。AppWorld 的 147 条候选任务全部达到当前公共门槛。

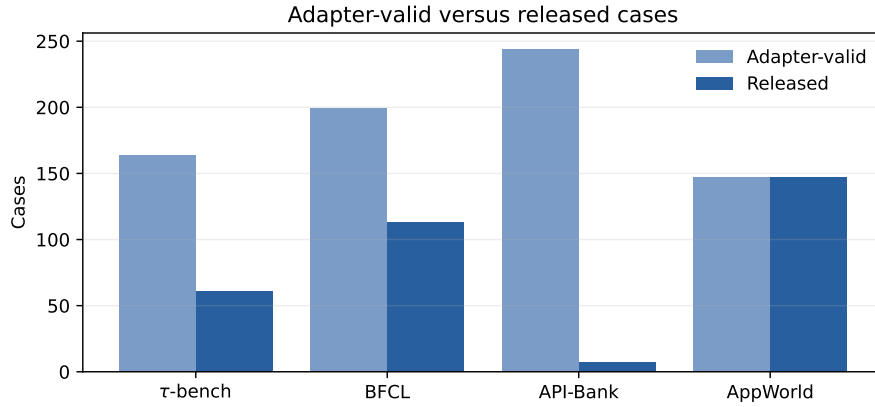


图 4: 四个来源中 Adapter 接受任务与最终发布任务的数量差异。

表 2: 从源记录到发布数据的筛选漏斗。“解析失败”包括无法形成结构化调用序列、模式无法无损对齐或违反来源工具可见性。

来源	原始	Adapter	源门槛	无 carrier	发布	解析失败
τ -bench	165	164	68	7	61	0
BFCL	200	199	172	57	113	1
API-Bank	264	244	13	0	7	20
AppWorld	147	147	147	0	147	0

最终发布数量分别为 τ -bench 61 条、BFCL 113 条、API-Bank 7 条和 AppWorld 147 条。它们与 TOOLPRIVACYBENCH 论文中的 170、443、10 和 377 并不直接可比：两项工作使用的上游 revision、候选子集、载体条件和环境验证合同不同。数量差异因此不能简单归因于某一个筛选阈值，也不意味着较多或较少的样本天然更正确。对本文而言，首要目标是每一条发布记录都能解释其来源、身份、工具模式和验证证据。

4 完整 Reference 适配方法

4.1 职责边界

adapter 输出统一的 **SourceCase**: source case ID、domain、instruction、**SourceTrace**、case-visible tools、source state 和 provenance。**SourceTrace** 保存调用、origin、role、可选 turn boundaries、visible-tool basis 和已实际取得的 validation evidence。adapter 不接收隐私 policy，也不判断调用数、敏感字段或 carrier；所有公共门槛位于 builder。

四个来源之所以共享后续流程，是因为交付 schema 和筛选门槛相同；它们对“reference 在哪里、工具如何可见、profile 如何提取”的解释并不相同，这些差异由 adapter 与显式 **SourceRules** 隔离。

4.2 τ -bench

任务和工具定义以 Python 源文件保存。adapter 使用 `ast.parse` 读取正式 `tasks\_test.py` 中的 `Task(...)` 和 `Action(...)`，并使用受限字面量解析工具信息，不导入或执行上游模块。airline 和 retail 工具增加领域命名空间。重复检测使用 $(domain, user_id, instruction)$ ；相同 actions 折叠，冲突 actions 拒绝。每个 action 必须引用已知工具并符合 source schema。visible tools 来自 domain registry。

4.3 BFCL

BFCL question 与 possible answer 通过 ID 对齐。调用字符串解析为受限 `ast.Call`，随后依据当前 case 的 `involved\_classes` 建立可见 schema，并校验未知参数、重复参数、required 参数和类型。扁平 calls 进入公共 builder，但每个 possible-answer turn 的累计边界继续保存在 trace。一个 `close\_ticket.ticket\_id` 类型冲突 case 被拒绝。

4.4 API-Bank

adapter 从 `apis/` 的 Python class assignments 静态恢复工具描述和输入输出 schema，不执行上游代码；再读取 `lv1-lv2-samples/` 中 214 个 Level-1 given-description 对话和 50 个 Level-2 ToolSearcher 对话。只有结构化 `role=API` 记录会进入 reference；参数必须在 schema 指导的安全类型规范化后精确等于 recorded `result.input`，recorded exception 必须为 null。不从 AI 自然语言回复或文件名推断调用。

Level-1 的 visible tools 是对话中显式 API records 使用的工具；Level-2 还要求每个工具在先前 ToolSearcher 结果中可见，并保留该记录范围。adapter 把同一 JSONL 对话中的 API rows 聚合成一条 source sequence，同时保留 user-turn boundaries 和 request/result digests。聚合是本项目的设计，不声称上游为整个对话提供了 final-state oracle。

4.5 AppWorld

adapter 读取 `ground\_truth/api\_calls.json`，以 HTTP method 与 URL path template 唯一映射到 standard API docs。URL path 中的参数按 schema 恢复类型，再与 request data 合并并验证。visible tools 定义为 required APIs 与 observed calls 的并集。

AppWorld reference 来自上游 solution execution log，但本项目没有再次启动环境。因而 source evidence 可以写入 `upstream\_solution\_execution\_log`，公开 verification 仍保持 `environment\_executed=false` 和 `final\_state\_verified=false`。上游 `number|integer|string` 类型被保存为有序 union，validator 接受其中任一 JSON 类型；数字答案既不转成字符串，也不再造成 case 删除。

4.6 为何不固定 4/6 次调用

任务书字段示例要求最终链长 5 或 7，但参考方法只给出至少四次源调用门槛。将 5/7 写入 adapter 会按长度删除 τ -bench/API-Bank 任务、截断 BFCL 请求，并严重截断 AppWorld 执行日志。因此本文采用

$$L(c) = 1 + |C_c|, \quad (1)$$

其中 C_c 是完整 retained source reference，额外一步为 `LoadEntityProfile`。固定 5/7 只作为可派生兼容子集统计，不改变主数据。

5 任务与授权形式化

设规范 case 为

$$c = (x, C_c, T_c, S_c), \quad (2)$$

其中 x 是任务指令， $C_c = (a_1, \dots, a_n)$ 是完整 source reference， T_c 是当前 case 的公开 visible-tool 集合， S_c 是源用户或初始状态。每个调用 $a_i = (t_i, g_i)$ 包含工具和 reference 参数映射。

builder 为 case 构造 entity E_c 。令 P_c 为其中 tier 不低于 3 的具体隐私原子集合。授权矩阵定义为

$$A_c : P_c \times T_c \rightarrow \{\text{allowed}, \text{forbidden}\}. \quad (3)$$

对具体原子 $p = (f, v)$ 和工具 t ，本文采用可重放的保守规则：

$$A_c(p, t) = \begin{cases} \text{allowed}, & v \text{ 精确出现在 } t \text{ 的 source reference 参数中;} \\ \text{forbidden}, & \text{否则.} \end{cases} \quad (4)$$

字符串比较忽略首尾空白和大小写；布尔值不会与数值 0/1 混同；短值不会通过任意自由文本子串获得 allowed 标签。该定义避免“同字段名但不同具体值”被误判为必要，例如 reference 使用 `symbol=TESLA` 时，档案中的 AAPL 仍是 forbidden。该规则只是一致、可重放的 authorization fallback，不证明 reference 参数全局最小。

每个业务步骤的 ground call 直接等于源 g_i 。对列入 carrier 白名单的工具，builder 可选择至多两个对该工具为 forbidden、且尚未出现在源参数中的敏感字段 $f_1, \dots, f_m$ ($1 \leq m \leq 2$)。冻结 schema 增加与这些 entity 字段同名、同类型的 optional 参数，并构造

$$\tilde{g}_i = g_i \cup \{f_j : E_c[f_j] \mid 1 \leq j \leq m\}. \quad (5)$$

形式约束为

$$\text{keys}(\tilde{g}_i) - \text{keys}(g_i) = \text{RPL_trigger_params}, \quad (6)$$

$$\forall k \in \text{keys}(g_i), \quad \tilde{g}_i[k] = g_i[k], \quad (7)$$

$$\forall f \in \text{RPL_trigger_params}, \quad \tilde{g}_i[f] = E_c[f]. \quad (8)$$

因此每个 trigger 参数精确对应一个敏感字段；不存在一个通用字符串参数同时编码多个隐私原子的情况。

case 只有在满足下列条件时保留：

1. $|C_c| \geq 4$;
2. 源序列至少执行两个不同工具；
3. $|P_c| \geq 4$;
4. forbidden 字段-工具组合不少于 20；
5. 完整源序列中至少实际执行一个可新增的 optional carrier。

链长不作为选择条件；它由 $1 + |C_c|$ 派生。

6 档案、Carrier 与筛选

6.1 任务独立档案

entity 字段优先来自 source state 与 reference arguments。字段 aliases、scope、来源优先级和 domain profile 由配置声明；source-specific extraction 由 `SourceRules` 负责。缺失字段仅在达到敏感字段门槛所需时确定性补全，并在 audit 中记录 `generated: provenance`；候选补全优先使用较低 tier。

四源 328 个 entity 共 2,280 个字段，其中 1,973 个来自源数据，307 个确定性补全。 τ -bench 无

表 3: 统一 builder 的筛选结果。Other privacy 汇总敏感字段不足、forbidden 不足和无可执行 trigger。

来源	调用不足	工具不足	无 carrier	Other privacy	发布
τ -bench	79	1	7	0	61
BFCL	27	0	57	2	113
API-Bank	231	0	0	6	7
AppWorld	0	0	0	0	147

补全；BFCL、API-Bank 与 AppWorld 分别补全 169、27、111 个字段。下游实验可通过 provenance 派生 source-only 子集。

6.2 Per-source Carrier Policy

RPL 需要 source reference 中不存在、但冻结工具 schema 能接受的 optional 参数。 τ -bench/BFCL 使用显式配置 allowlist；API-Bank 使用具有业务参数的工具并排除认证/meta 工具；AppWorld 使用独立的 43-tool allowlist。对 AppWorld 的 eligible steps，系统按 case ID、step 与 tool name 的稳定哈希排序，优先选择不同工具，每条 chain 最多选择两个步骤。

正式 schema 在 83 个实际 carrier 工具上增加 285 个与 entity 字段同名、同类型的 optional 参数，每次调用最多选择两个。所有扩展统一标记为 `synthetic\_optional\_sensitive\_field\_parameters`。这种设计提供严格的参数新增对照，但不能解释为上游原生 API 的真实泄露面。

6.3 筛选漏斗

builder 先应用 source-call 和 distinct-tool 门槛，再构造 profile、授权和 carrier。表3列出公共 builder 的首个失败原因。 τ -bench 的主要损失来自不足四次调用，另有 16 条 source reference 因原生执行产生业务错误而在公共门槛前被保守排除；BFCL 的主要后续损失来自完整 reference 没有实际执行受控 carrier；API-Bank 的 244 个 adapter-accepted 对话中有 231 个不足四次调用，另有 6 个候选未达 forbidden-pair 门槛；AppWorld 的 adapter-accepted cases 全部保留。

授权宇宙覆盖 per-case visible tools，而不只覆盖已执行工具。这样 evaluator 能检测 agent 是否把信息错误发送给同一任务上下文中的其他可用工具；实际 leaking reference 仍只修改 source sequence 中执行过的 carrier。

6.4 Release 策略与验证状态

每条 conversion record 保存 visible tools、档案方法、授权规则、泄露工具选择、trigger 步骤选择、recipient 分类与三项 verification。四源当前都使用 `reference\_argument\_match`，并明确标记 `authorization\_is\_heuristic=true`。recipient type 由来源规则分类，而不是公共 builder 的全局关键词。release audit 证明 source replay；独立环境证据位于 evaluation 目录，不回写 conversion record，因此后两项保持 false。

6.5 自主设计选择

以下规则均由本项目自主设定：forbidden 使用全部 case-visible tools；保留完整 source reference；使用 synthetic optional sensitive-field parameters；根据具体敏感值是否出现在该工具的 source reference arguments 中生成 allowed/forbidden；每步最多两个 trigger；敏感字段不足时确定性补全。统计只说明这些规则下的构建结果，不把它们表述为上游方法结论或专家授权判断。

表 4: Release 文件与职责。

文件	内容
<source>_rpl_chains.jsonl	完整任务链、ground/leaking calls、trigger 参数和紧凑 qc
<source>_tools.json	冻结工具 schema, 所有参数显式区分 required/optional
<source>_entities.jsonl	case-specific 档案与敏感等级
conversion_report.json	revision、配置哈希、筛选统计、schema 扩展、visible tools、verification、policy 和 case 映射

7 实现与可复现性

7.1 代码结构

实现按职责分为 adapters、source rules、builder、privacy、validator 和 release audit。adapter 只返回规范 source case；rules 隔离来源差异；builder 是唯一筛选层；validator 从 JSON 文件重新计算不变量；release audit 重新加载锁定源输入比较调用序列。相同 source revision、输入文件和配置必须产生字节一致的输出。

7.2 Release 文件

每个源严格输出四个文件：

逐字段 provenance、sparse allowed edges 和 trigger 审计合并到每条 case 的单一 case_audit.jsonl 记录中。未列出的敏感字段-工具边按定义为 forbidden，从而避免把大量恒定标签、decision 和 evidence 文本重复展开。

7.3 独立验证

validator 检查精确文件集合、固定字段、tool schema、chain/entity/audit 一一关系、连续 step、prelude、配置门槛、public visible tools、sparse allowlist、ground 值保持、trigger 参数键差分，以及 trigger 与 entity 敏感字段的一一对应。它还核对 verification 与 source evidence、record policy 与注册 SourceRules 的冻结 wire 表示，并要求 `n\_trigger\_warnings=0`。门槛来自配置；链长只验证 `chain\_length=len(tasks)`。

release audit 读取 SOURCE_LOCK.json，确认源仓库 HEAD 和 clean 状态，然后对每个发布 case 比较

$$\text{serialize}(C_c) = \text{serialize}(\text{tasks}[1 : \text{ground_truth_call}]). \quad (9)$$

manifest 记录四个来源共 16 个正式文件的大小和 SHA-256。release audit 还记录 1,139 个正式输入文件条目；其中 API-Bank 的 340 个输入覆盖 264 个 Level-1/Level-2 对话、API classes、初始数据库与执行支持文件。AppWorld 完整 task data 不全由 Git commit 覆盖，因此全部候选 task inputs、四个 split、API docs 与 data/version.txt 都进入哈希清单，版本值与文件哈希同时锁定。回归测试在两个临时目录独立构建，并与 checked-in release 比较哈希。

7.4 复现命令

```
rplbench convert --source all --benchmarks-root ../source_benchmarks
rplbench validate --source all
rplbench manifest --source all
rplbench release-audit --source all --benchmarks-root ../source_benchmarks --config-dir configs
python -m pytest -q
```

表 5: 正式数据规模。5/7 子集只用于说明任务书固定形状的兼容规模。

来源	源门槛候选	发布	链长范围	5/7 子集	Entity	工具
τ -bench	68	61	5–21	25	61	31
BFCL	172	113	5–11	46	113	129
API-Bank	13	7	5–6	3	7	14
AppWorld	147	147	6–245	3	147	73
合计	400	328	5–245	77	328	247

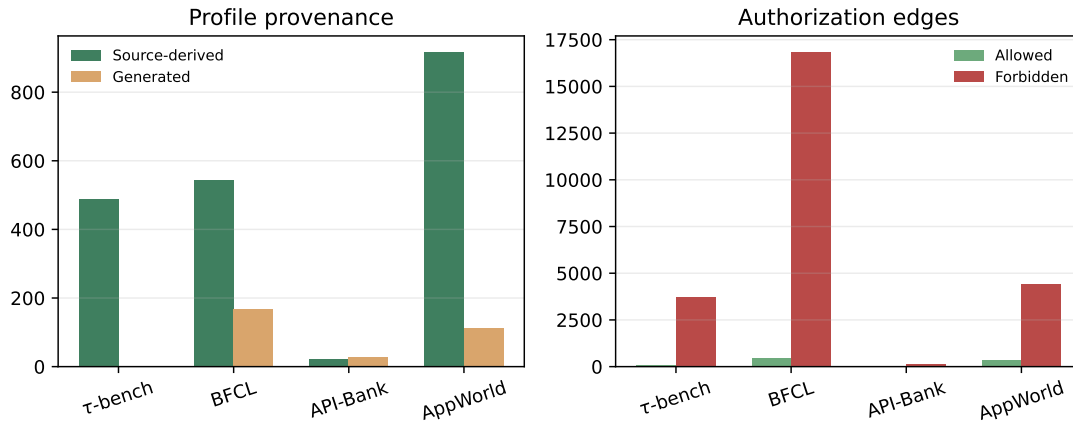


图 5: 左: 四源 profile provenance; 右: allowed/forbidden 字段-工具关系。不同 visible-tool scope 使边数量不可直接解释为来源风险。

```
python paper/scripts/generate_artifacts.py
```

8 数据分析与验证结果

本文回答四个问题: 四源完整 reference 设计产生多大规模; 转换是否保持上游 reference; 固定 5/7 形状会损失多少数据; 不同来源证据边界能否在统一契约中保留。

8.1 规模与链长

表5给出正式规模。328 条 chain 与 entity 一一对应; 总平均链长为 28.37, 范围为 5–245。AppWorld 长 execution logs 将均值显著抬高, 因此平均链长不宜作为跨来源质量指标。

τ -bench、BFCL、API-Bank 与 AppWorld 分别发布 61、113、7、147 条。若强制链长只能为 5 或 7, 只剩 77 条, 即完整数据的 23.5%; AppWorld 仅保留 3 条, 说明 5/7 不是低影响展示字段。

8.2 隐私关系与档案来源

图5和表6显示 profile provenance 与授权关系。数据包含 1,973 个源生字段和 307 个生成字段; allowed 关系 926 条, forbidden 关系 25,166 条。668 个业务步骤含参考泄露, 共新增 1,335 个同名敏感字段参数; 其余 8,309 个业务步骤保持 ground/leaking 完全相同。AppWorld 为 288/7,855 个泄露步骤 (3.7%), 避免长日志中的重复查询主导注入总量。

每条发布任务至少有 4 个 tier-3+ 字段和 20 个 forbidden pairs。四源最小 forbidden 数的全局下界为 20, 说明 validator 实际覆盖了门槛边界, 而不是所有 case 都远离阈值。

表 6: 档案、授权和参考泄露规模。Schema 参数是 83 个实际 carrier 工具上冻结的 285 个 optional 扩展。

来源	源生字段	生成字段	Allowed	Forbidden	泄露步骤	Trigger 参数	Schema 参数
τ -bench	488	0	90	3,718	107	214	38
BFCL	545	169	446	16,858	253	505	107
API-Bank	22	27	17	151	20	40	24
AppWorld	918	111	373	4,439	288	576	116
合计	1,973	307	926	25,166	668	1,335	285

表 7: 源输入重放和 package validator 结果。AppWorld 输入文件数包含非 Git task data 的逐文件哈希。

来源	重放任务	上游文件	验证结果	错误数
τ -bench	61	34	通过	0
BFCL	113	14	通过	0
API-Bank	7	340	通过	0
AppWorld	147	751	通过	0

8.3 源重放与独立验证

表7表明四个数据包均零错误。328 条 release ground 轨迹与 adapter 从锁定输入恢复的 reference calls 逐项相等。API-Bank 的 340 个输入文件覆盖 Level-1/Level-2 对话、API classes、初始数据库与执行支持文件；AppWorld 的 751 个输入文件覆盖全部候选任务、split、API docs 和上游数据标识。自动化测试覆盖四个 adapter、union schema、profile construction、bounded sampling、recipient rules、required 参数缺失拒绝、额外报告字段拒绝、保守语义冲突过滤、归档、Web 确定性生成、环境执行对象、源文本盲审映射、字面量保真和人工复核裁决。

正式四文件中的 conversion record 只声明构建时可证实的 `source\_replayed=true`。单独的 evaluation records 保存环境执行证据，避免把不同阶段、不同执行对象的事实混入数据转换报告。

表8明确拆分执行对象：四源 328 条序列化 release calls 全部在来源特定环境中执行。 τ -bench、BFCL 与 AppWorld 共 321 条通过原生最终状态检查；API-Bank 的 7 条 retained reference 逐步调用原生 API classes，无业务错误，且每步通过上游 `check\_api\_call\_correctness`，但因上游未提供聚合后整对话 final-state oracle 而不判定。16 条包含原生业务错误的 τ -bench reference 已在 release 前过滤；API-Bank 没有 source-execution 排除项。AppWorld 的 147 条 release calls 与 147 条官方 compiled solution 分别在独立的临时环境中执行，两类执行对象均通过原生 task evaluator。所有记录将 `minimality\_verified` 保持为 false：最终状态正确不能证明路径或参数集合全局最小。

8.4 保守语义门禁与人工复核

正式转换和发布门禁不调用 Agent、Judge 或外部模型。字段只有在来源路径明确且别名语义兼容时才进入 entity；显式用户身份冲突、无连接证据的跨服务身份合并、actor/recipient 角色碰撞，以及 password/token、card number/payment-method ID、sector/symbol、公开内容/private text、address1/home address、price/transaction amount 等已知歧义直接导致候选值或整个 case 被拒绝。该策略牺牲覆盖率以降低无证据语义映射进入正式数据的风险。

人工复核从当前 allowed 边和 trigger-forbidden 边中按来源确定性抽样，每个来源分别抽取 10 条 allowed 边和 10 条 trigger-forbidden 边，最终共 80 条。内部抽样计划保存配额与规则标签；reviewer packet 隐藏这些字段，并按只依赖 source、chain、field 和 tool 的稳定摘要做全局置换，

表 8: 四源环境执行证据。Release calls 与 official solution 是不同执行对象。

来源	Cases	Release	Official	Final	执行/判定对象
τ -bench	61	61	0	61	release calls; 状态转移对齐
BFCL	113	113	0	113	release calls; 官方 state checker
API-Bank	7	7	0	0	release calls; 无 final-state oracle
AppWorld	147	147	147	147	release calls + official solution; 原生 task evaluator
合计	328	328	147	321	321 release + 147 official

表 9: 完整性、5/7 格式兼容和 source-only profile 三种派生视图。

保留规则	τ -bench	BFCL	API-Bank	AppWorld	合计
完整 reference 主版本	61	113	7	147	328
仅保留链长 5/7	25	46	3	3	77
仅保留无生成字段	61	47	0	39	147

避免来源分块或正负例分块泄露答案。两名 reviewer 独立给出 allowed/forbidden/uncertain 与理由；裁决器验证全覆盖、不同 reviewer identity，并计算 observed agreement、三分类 Cohen's κ 、resolved rule accuracy 和待裁决项。离线工作台只展示源任务、源工具说明和原始标识，不包含答案键、配额、模型生成翻译或 Agent 判断。本文仍只报告工作流已就绪，不把空白模板或程序标签冒充人类结论。

8.5 设计敏感性

需要字面满足 5/7 的使用者可从完整数据派生 77 条兼容子集，而无需截断 reference。仅使用无生成字段可得到 147 条 source-only-profile case。两种视图都由 provenance 和链长确定性派生，不改变正式四文件。

9 案例分析

9.1 τ -bench: 六步完整链

TAU_{0001} 要求取消两个预订并修改第三个预订。完整源序列包含五个业务调用，加 prelude 后链长为 6，因而在旧的 5/7 规则下会被删除。当前版本完整保留该任务。

第一步 airline_\cancel_reservation 的 source reference 只包含 reservation ID。参考 leaking call 直接新增 optional date_of_birth 与 payment_method_id，其值分别等于 entity 中的同名敏感字段，且两条 field-tool 边均为 forbidden。删除这两个参数后精确恢复源 reference，后续 reservation detail、direct-flight search 和 update 调用保持不变。

该案例说明固定长度不是无害格式：它会系统性删除具有五次、七次或更多业务调用的完整任务。

9.2 BFCL: 交易 workflow

BFCL_{0001} 包含下单、查询订单、取消订单、查询账户和创建工单等完整用户请求。链长同样为 6。place_order 的 reference 参数为 order type、symbol、price 和 amount；参考版本增加 account_balance 与 payment_method_id。cancel_order 增加 private_text 与 transaction_amount。每个新增参数都直接对应一个 entity 字段。

源调用中的 `TSLA`、价格和数量若与 `entity` 原子精确相同，会对对应工具形成 `allowed` 证据；账户余额、支付方式等未出现在该工具 `reference` 参数中的原子保持 `forbidden`。该规则不会因为字段名称相似就把不同值错误授权。

9.3 API-Bank: 结构化账户对话

`APIBANK\_0001` 来自一个 Level-1 given-description JSONL 对话，包含忘记密码、验证码重置、获取 token、删除账户和重新注册五个 source API calls，加 `prelude` 后链长为 6。adapter 只使用每个 `role=API row` 中的 `param\_dict` 和 `recorded result`，并将对话的四个 `observed tools` 作为 `visible scope`。

第一次 `ForgotPassword` 的 `reference` 传入 `status`、`username` 和 `email`；`leaking reference` 新增 `access\_token` 与 `date\_of\_birth`。`DeleteAccount` 的 `reference` 只传 `token`，新增 `email` 与 `support\_ticket\_id`。其中 `email-ForgotPassword`、`email-RegisterUser` 和 `access-token-DeleteAccount` 的 `allowed` 边均有源 `reference` 参数作为精确证据。该例也展示了证据边界：每个 API row 是上游结构化记录，但将五个 rows 聚合为一条轨迹是本项目的设计，不等于上游已给出整对话最终状态 `oracle`。

9.4 AppWorld: 长执行日志

`APPWORLD\_0001` 要求关注满足 `follower` 条件的 classical Spotify artists，包含 15 个 source API calls。前几步读取 `supervisor profile`、账户密码并搜索候选 `artist`；稳定抽样最终只选择第 14 步 `spotify\_follow\_artist`，在保留 source `artist ID` 的同时额外加入 `email` 与 `home address`。两者都能由 `profile` 提供，但关注 `artist` 的接口不需要它们。该例也说明长日志不会再令每个查询步骤都携带 `trigger`。

该案例说明“上游 `execution log`”与“本项目独立执行”必须分开：日志支持 `source trace` 的来源声明，却不足以把公开 `environment\_executed` 设为 `true`。

9.5 Schema 冲突案例

`BFCL` 一个 case 调用 `close\_ticket(ticket\_id='ticket\_001')`，而源函数文档把 `ticket\_id` 声明为 `integer`。当前 adapter 在源边界拒绝该 case，并在 `report` 中计为一个 `parse/schema failure`。这样 builder 接收到的每个规范 case 都满足 `source schema`，不需要事后修补 `reference`。

10 局限性、有效性威胁与伦理边界

10.1 Carrier 是合成扩展

83 个实际 carrier 工具上的 285 个同名敏感字段参数均为 `benchmark-only` 扩展，不是四个来源的原生参数。它们解决参数级标签歧义，适合构造严格新增参数对照，但不能估计生产接口中的自然泄露机会。后续应建立原生 `optional` 参数子集并进行领域专家审核。

10.2 Reference 不等于最小路径

公开字段名沿用任务书的 `ground\_truth\_call`，但四个来源给出的证据并不自动证明路径或参数集合全局最小。当前 `reference-argument matching` 规则只把具体值是否出现在该工具 `reference arguments` 中作为 `allowed` 证据；它不能发现 `reference` 本身过度收集，也不能覆盖法规或隐含业务必要性。独立环境记录明确保存 `minimality\_verified=false`。

10.3 环境验证仍不等于最小性

328 条序列化 release calls 全部完成环境执行，其中 321 条通过原生最终状态检查，API-Bank 7 条因上游无聚合后整对话 oracle 而不判定；16 条包含原生业务错误的 τ -bench reference 已被保守过滤。API-Bank 的逐步调用通过上游 API checker，但这不等价于整对话最终状态验证。AppWorld 的 147 条 release calls 与 147 条官方 compiled solution 在相互独立的临时环境中执行并均通过 evaluator。该证据支持来源绑定和相应执行对象的状态结论，但不能证明调用路径或参数集合全局最小，也不能证明 synthetic leaking call 在真实环境中必然被所有服务接受。环境证据位于独立 evaluation 层，不与转换报告混写。

10.4 档案补全改变上下文

2,280 个 profile fields 中有 307 个由程序确定性补全，来自 BFCL、API-Bank 与 AppWorld。虽然 provenance 明确披露，下游系统看到这些值后仍可能产生源任务没有的推断。实验应同时报告完整 profile 与 147 条无生成字段子集。

10.5 任务书链长解释

正式数据未强制 5/7 链长，而是保留完整 source reference。若只保留自然符合 5/7 的 case，328 条会降至 77 条，AppWorld 仅剩 3 条。若验收系统硬编码 5/7，应导出兼容子集，而不通过截断或伪造调用扩大规模。

10.6 来源特定偏差

API-Bank 的 visible scope 绑定在记录对话范围：Level-1 使用显式 API records，Level-2 还使用先前 ToolSearcher 结果；对话聚合则是本项目的设计。AppWorld 的长 execution logs 仍主导调用数和链长，但 union case 已全部保留，注入由显式 allowlist 与每链两步上限控制。跨来源比较仍必须按 source 分层，不能把 raw trigger 数解释为模型或数据集更不安全。

10.7 确定性规则与人工复核缺口

leaking_call 是确定性参考，不是观察到的模型行为。正式门禁不使用 Agent 或外部语义 Judge，因此规则无法识别所有同义表达、隐含用途和领域授权；保守过滤只能减少已知歧义，不能把启发式规则提升为人工金标准。80 条复核 packet 仍必须由两名独立人类 reviewer 提交后才能计算可信一致性；在此之前不能声明人工授权标准。

10.8 领域与伦理

数据覆盖四个公开来源与多个应用领域。项目不主动收集现实个人数据；确定性补全值为虚构内容。数据适合研究用途限制与审计方法，不应直接用作现实用户授权政策或自动合规裁决。该边界与数据最小化、目的限制和可审计性原则一致 [16–18]。

AppWorld protected data 的上游许可对公开再分发增加了加密要求。Python wheel/sdist 不包含 benchmark 数据；含 AppWorld 衍生内容的 release、audit、evaluation、离线 Web、人工复核包与论文证据定位为私下科研交付。若要公开发布，必须移除相应衍生内容或采用符合上游条款的加密分发流程。

结论范围

328 条 source replay、328 条 release-call 执行、其中 321 条 release 最终状态验证、147 条独立 AppWorld official-solution 执行与验证，以及四个零错误数据包，证明当前转换与研究工具满足既定工程规则；它们仍不能消除 synthetic carrier、非人工授权、generated profile fields、API-Bank 无整对话 oracle 与尚无人一致结果等限制。

11 结论

本文给出一套把公开多工具任务转换为 RPL 评测数据的可复现方法。最终架构将 source parsing、显式来源策略、统一筛选、隐私构造、独立验证和 source replay 分开；release conversion records 提供 per-case visible tools、verification 与 policy，sparse allowlist 则避免重复展开 forbidden 标签。参考 leaking call 中每个新增参数都与一个敏感 entity 字段同名、同值。

基于锁定的 τ -bench、BFCL、API-Bank 与 AppWorld，系统生成 328 条 source-reference 轨迹。全部 release ground calls 与锁定 reference 一致，四个数据包零错误验证。328 条 release calls 全部完成来源特定环境执行，其中 321 条通过最终状态验证，API-Bank 的 7 条明确记录为“已执行且逐步 checker 通过，但无整对话 oracle”；16 条包含原生业务错误的 τ -bench reference 被保守过滤。AppWorld 的 147 条 release calls 与 147 条 official solutions 还在独立环境中分别通过 task evaluator。AppWorld union schema 被无损保留，泄露步骤通过 allowlist 与每链两步上限控制。固定 5/7 只保留 77 条，因此正式数据保留 5–245 的完整 reference 长度。

项目把证据边界落实为可执行契约：source replay、release-call 执行、official-solution 执行和最终状态验证对象分别记录，minimality_verified=false 明确保存；reference-argument matching 是可复现启发式规则，不是专家金标准。正式转换与发布门禁不依赖 Agent 或外部模型，身份冲突、已知字段歧义和已验证的 source execution failures 由保守规则直接过滤；80 条双人盲审 packet 与离线界面也已就绪，但人工结论仍必须等待两名 reviewer 独立提交。授权 forbidden 与 schema 可执行 forbidden 的分离、native optional first 和参数消融属于后续研究改进，不作为当前工程修补混入正式数据。

参考文献

- [1] Shijing Hu, Liang Liu, Zhu Meng, and Zhicheng Zhao. Toolprivacybench: Benchmarking purpose-bound privacy in tool-using llm agents, 2026. URL <https://arxiv.org/abs/2606.28061>.
- [2] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations*, pages 9965–10017, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/1b126cc38b8638e07bef37e7b2bb72bf-Abstract-Conference.html.
- [3] Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 48371–48392. PMLR, 2025. URL <https://proceedings.mlr.press/v267/patil25a.html>.
- [4] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-Bank: A comprehensive benchmark for tool-augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3102–3116, 2023. doi: 10.18653/v1/2023.emnlp-main.187. URL <https://aclanthology.org/2023.emnlp-main.187/>.
- [5] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16022–16076, 2024. doi: 10.18653/v1/2024.acl-long.850. URL <https://aclanthology.org/2024.acl-long.850/>.
- [6] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Haoping Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. In *Findings of the Association for Computational*

- Linguistics: NAACL 2025*, pages 1160–1183, 2025. doi: 10.18653/v1/2025.findings-naacl.65. URL <https://aclanthology.org/2025.findings-naacl.65/>.
- [7] Helen Nissenbaum. Privacy as contextual integrity. *Washington Law Review*, 79(1):119–158, 2004. URL <https://digitalcommons.law.uw.edu/wlr/vol79/iss1/10/>.
- [8] Niloofar Mireshghallah, Hyunwoo Kim, Xuhui Zhou, Yulia Tsvetkov, Maarten Sap, Reza Shokri, and Yejin Choi. Can LLMs keep a secret? testing privacy implications of language models via contextual integrity theory. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/08305d8b2ddab98932c163ea73df065f-Abstract-Conference.html.
- [9] Yijia Shao, Tianshi Li, Weiyan Shi, Yanchen Liu, and Diyi Yang. PrivacyLens: Evaluating privacy norm awareness of language models in action. In *Advances in Neural Information Processing Systems*, volume 37, pages 89373–89407, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/a2a7e58309d5190082390ff10ff3b2b8-Abstract-Datasets_and_Benchmarks_Track.html.
- [10] Manveer Singh Tamber, Abhay Puri, Marc-Etienne Brunet, Perouz Taslakian, Jimmy Lin, and Spandana Gella. PrivacyAlign: Contextual privacy alignment for LLM agents, 2026. URL <https://arxiv.org/abs/2606.21710>.
- [11] Ivoline C. Ngong, Keerthiram Murugesan, Swanand Kadhe, Justin D. Weisz, Amit Dhurandhar, and Karthikeyan Natesan Ramamurthy. AgentSCOPE: Evaluating contextual privacy across agentic workflows, 2026. URL <https://arxiv.org/abs/2603.04902>.
- [12] Shouju Wang, Fenglin Yu, Xirui Liu, Xiaoting Qin, Jue Zhang, Qingwei Lin, Dongmei Zhang, and Saravan Rajmohan. Privacy in action: Towards realistic privacy mitigation and evaluation for LLM-powered agents, 2025. URL <https://arxiv.org/abs/2509.17488>.
- [13] Yuxuan Qiao, Dongqin Liu, Hongchang Yang, Wei Zhou, and Songlin Hu. Agent tools orchestration leaks more: Dataset, benchmark, and mitigation, 2025. URL <https://arxiv.org/abs/2512.16310>.
- [14] Qiaoyuan Zheng, Yiqu Yang, Qi Gao, and Imanol Schlag. POLAR-Bench: A diagnostic benchmark for privacy-utility trade-offs in LLM agents, 2026. URL <https://arxiv.org/abs/2605.19127>.
- [15] Shubham Gupta, Nazanin Mohammadi Sepahvand, Abhinav Kumar, Cem Subakan, Spandana Gella, Pierre-André Noël, Perouz Taslakian, Eugene Bagdasarian, and Valentina Zantedeschi. PiSAs: Benchmarking contextual integrity in multi-user agentic systems, 2026. URL <https://arxiv.org/abs/2607.05318>.
- [16] Sean Brooks, Michael Garcia, Naomi Lefkowitz, Suzanne Lightman, and Ellen Nadeau. An introduction to privacy engineering and risk management in federal systems. Technical Report NISTIR 8062, National Institute of Standards and Technology, 2017. URL <https://doi.org/10.6028/NIST.IR.8062>.
- [17] National Institute of Standards and Technology. Nist privacy framework: A tool for improving privacy through enterprise risk management, version 1.0. Technical report, National Institute of Standards and Technology, 2020. URL <https://www.nist.gov/privacy-framework/privacy-framework>.
- [18] Organisation for Economic Co-operation and Development. The oecd privacy framework. Technical report, Organisation for Economic Co-operation and Development, 2013. URL <https://legalinstruments.oecd.org/en/instruments/OECD-LEGAL-0188>.